

SeuGado — Leandro: painel admin, rotina e deploy

Clientes, fazendas e lotes no painel da equipe; rotina diária e entrega semanal; publicação

27/09/2026

Sua parte em uma frase: você liga o sistema de ponta a ponta. A equipe cadastra clientes, fazendas e lotes no painel; o sistema calcula todo dia com o satélite, entrega o plano no dia e hora que cada produtor escolheu, e tudo está publicado na internet.

Regras do MVP (ADR-025):

- O site é **só da equipe (admin)**. O produtor **nunca** entra no site; ele só conversa com o bot do Telegram.
- Não existe cadastro aberto: as contas da equipe são criadas pelo Kauê no Supabase.
- Todo usuário logado é admin e vê todas as fazendas.
- **Só Marandu em pastejo rotacionado.**

Você entrega:

#	Tarefa	Prioridade
L0	Backend base: /me, clientes e fazendas (PR pequeno, merge cedo)	primeiro, até 10h
L1	Identidade visual (Figma) + tema.css	1 h
L2	Login da equipe	
L3	Página “Clientes e fazendas” + página “Fazenda” (configuração e Telegram)	
L4	Lotes	
L5	Página “Plano da semana”	
L6	Rotina: cálculo diário, entrega semanal e recálculo (jobs/ciclo.py)	
L7	Deploy: Render, Vercel e GitHub Actions	pode ir para segunda

1. O SeuGado em um minuto

O pecuarista divide o pasto em **piquetes** e o gado em **lotes**. O capim tem uma altura certa para o lote entrar e uma altura em que ele precisa sair. O SeuGado estima a altura de cada piquete por satélite e clima **todo dia**. Uma vez por semana, no dia e hora que o produtor escolheu, o bot manda o **plano da semana**: “segunda: mova o lote Recria do Piquete 2 para o Piquete 6”. O plano respeita a rotina da fazenda: só propõe movimentação nos **dias de manejo** e nunca move mais **animais** por dia do que os funcionários dão conta.

Se no meio da semana chegam imagens novas e o plano muda em algo que o produtor ainda não fez, o bot pergunta se ele quer ver as mudanças e escolher entre o plano antigo e o novo.

Palavras que você vai usar

Termo	O que é
Cliente	O produtor (pessoa). Pode ter mais de uma fazenda
Lote	Grupo de animais que anda junto. É a unidade de manejo
Categoria	bezerro, bezerra (até ~1 ano), novilho, novilha (recria, ~1-3 anos), vaca, boi, touro
Peso médio	Define quanto o lote come ($\approx 2,2-2,4\%$ do peso por dia). Se ninguém sabe o peso, o sistema estima pela tabela de Unidade Animal (bezerros 112,5 kg · novilhos/novilhas 337,5 kg · vaca/boi 450 kg · touro 562,5 kg) e a recomendação fica com confiança menor
Lote indissolúvel	Lote que nunca pode ser juntado a outro (ex.: vacas com bezerro ao pé)
Funcionários disponíveis	Quantas pessoas fazem o manejo
Animais por funcionário por dia	Quantos animais cada pessoa consegue mover sozinha num dia
Dias de manejo	Dias da semana em que se pode mexer no gado (0 = segunda ... 6 = domingo)
Envio do plano	Dia da semana e hora em que o produtor quer receber o plano
Plano vigente / candidato	Vigente: o que o produtor está seguindo. Candidato: plano novo oferecido a ele, que ele pode aceitar ou recusar
Evento	Registro imutável de algo que aconteceu ("lote criado"). Nunca se edita o passado: grava-se um evento novo

Por que o sexo entra e a raça não? Macho e fêmea da mesma idade podem ter pesos bem diferentes, e o peso define o consumo. Quando ninguém sabe o peso, a tabela de reserva é por idade (não há fonte por sexo), com exceção do touro. A raça fica para depois do MVP. Animal por animal (brinco, pesagem) está fora do escopo (ADR-001).

Entradas e saídas: o que você recebe, de quem, e o que entrega

Esta seção é o seu contrato com o resto da equipe. **Nomes, tipos e formatos são exatamente estes**; se algo aqui parecer faltar ou estar errado, fale com o Kauê antes de inventar outro formato.

Resumo

	O quê	De quem / para quem	Como chega / sai
Recebe	login e sessão da equipe	Supabase Auth, via <code>api/auth.py</code> do Kauê	<code>Usuario</code> , <code>FazendaAutorizada</code> ; no front, <code>supabase</code> e <code>useFazenda()</code>
Recebe	fazenda selecionada no topo do site	esqueleto do Kauê	<code>useFazenda().fazenda</code> (e <code>fazendas</code> , <code>selecionar</code> , <code>recarregar</code>)

	O quê	De quem / para quem	Como chega / sai
Recebe	lista de piquetes	Ezequiel	GET /fazendas/{id}/piquetes → Piquete[] (id, nome, situacao, lote_atual_id...)
Recebe	peso de reserva por categoria	Kauê	peso_por_ua_kg(categoria: CategoriaAnimal) -> float (planner/estado.py)
Recebe	funções da rotina	Kauê, João, Leo	assinaturas na seção 5.2
Entrega	Fazenda (rotina)	João (via rotina)	carregar_fazenda(conn, fazenda_id) -> Fazenda
Entrega	usuário e lista de fazendas	o site todo (inclusive o Mapa do Ezequiel)	GET /me, GET /fazendas
Entrega	lotes gravados como eventos	banco → Kauê (consumo e posição) → João	lote_criado, lote_alterado, lote_dissolvido, manejo_confirmado
Entrega	plano vigente para a web	Ezequiel (mapa) e a sua página Plano	GET /fazendas/{id}/plano/atual → plano_para_dict(plano) ou 404
Entrega	recálculo imediato	Leo (quando o produtor diverge ou corrige alturas)	executar_ciclo(conn, fazenda_id, hoje=None, ingerir_satelite=True, enviar=True, atualizado=False) -> PlanoManejo
Entrega	rotina automática	todos	GitHub Actions de hora em hora → python -m seugado.jobs.ciclo --agenda

Fazenda no Python

Fazenda: a configuração (de seugado.core.models)

Campo	Tipo	Valores	Significado
id	UUID	—	Fazenda
nome	str	—	Nome
timezone	str	"America/Fortaleza"	Fuso
funcionarios_disponiveis	int	≥ 1	Pessoas que fazem o manejo
animais_por_funcionario_dia	int	≥ 1	Quantos animais cada pessoa consegue mover por dia
dias_preferenciais_manejo	tuple[int, ...]	0 = segunda ... 6 = domingo	Dias em que se pode mover gado
envio_plano_dia	int	0 = segunda ... 6 = domingo	Dia em que o produtor recebe o plano semanal
envio_plano_hora	int	0 ... 23 (hora local)	Hora em que recebe o plano
ativo	bool	—	Fazenda ativa

Em JSON (rotas /fazendas), os mesmos campos sem ativo, mais cliente_id e cliente_nome; dias_preferenciais_manejo vira lista ([0, 3]) e os ids viram texto.

Lote que você devolve (JSON), campo a campo

Campo	Tipo	Exemplo	Origem
id	texto (UUID)	"2222...0001"	estado_lote.lote_id
nome	texto	"Recria"	estado_lote.nome
indissoluvel	booleano	false	estado_lote.indissoluvel
composicao	lista de {categoria, n_animaais, peso_medio_kg, origem_peso}	[{"categoria": "novilho", "n_animaais": 150, "peso_medio_kg": 337.5, "origem_peso": "ua_tabela"}]	estado_lote.composicao (JSON)
piquete_atual_id / piquete_atual_nome	texto ou null	"...", "Piquete 2"	estado_lote.piquete_atual_id + estado_piquete.nome
desde	texto ISO ou null	"2026-09-25"	estado_lote.desde
peso_vivo_total_kg	número	50625.0	estado_lote.peso_vivo_total_kg
n_animaais_total	número	150	soma de n_animaais

origem_peso: "produtor" (informado) ou "ua_tabela" (estimado).

O plano que você mostra

PlanoManejo: o plano da semana

Tipo Python em `seugado.contratos`; em JSON via `plano_para_dict` / `plano_de_dict`; em TypeScript em `frontend/src/lib/tipos.ts`. Exemplo completo: `tests/fixtures/plano_exemplo.json`.

Campo	Tipo (Python)	Em JSON	Significado
id	UUID	texto	Id do plano
fazenda_id	UUID	texto	Fazenda dona do plano
data_geracao	datetime (UTC, com fuso)	"2026-09-28T08:00:00+00:00"	Quando foi gerado
data_inicio	date	"2026-09-28"	Primeiro dia do plano (hoje)
horizonte_dias	int	7	Sempre 7
movimentacoes	tuple[Movimentacao, ...]	lista	Ordenadas por (data, lote_nome)
alertas	tuple[Alerta, ...]	lista	Ordenados por (data, tipo, texto)
pedidos_validacao	tuple[PedidoValidacao, ...]	lista	Ordenados pelo nome do piquete
piquetes	tuple[ResumoPiquete, ...]	lista	Um por piquete ativo, em ordem de nome

Movimentacao: “mover o lote X do piquete A para o B no dia D”

Campo	Tipo	Em JSON	Significado
id	UUID	texto	Id da movimentação; é o <code>entidade_id</code> do evento <code>manejo_recomendado</code> e o que vai nos botões do bot
data	date	"2026-09-28"	Dia da movimentação (sempre dia de manejo preferido)
lote_id / lote_nome	UUID / str	texto	Lote que muda
piquete_origem_id / piquete_origem_nome	UUID None / str None	texto ou null	De onde sai
piquete_destino_id / piquete_destino_nome	UUID / str	texto	Para onde vai
altura_destino_cm	float (1 casa)	número	Altura prevista do destino no início do dia
altura_entrada_alvo_cm	float	número	Altura ideal de entrada do capim do destino
altura_origem_cm	float None	número ou null	Altura prevista da origem no início do dia
altura_saida_alvo_cm	float None	número ou null	Altura de saída do capim da origem
dias_previstos	int	número	Dias que o lote deve ficar no destino
motivo	str	texto	Frase pronta, em português, para o produtor
confianca	Confianca	"alta" "media" "baixa"	Confiança da recomendação
motivo_confianca	str	texto	Frase pronta: o dado mais fraco por trás da recomendação

Alerta

Campo	Tipo	Em JSON	Significado
tipo	TipoAlerta	um de: <code>sem_piquete_apto</code> , <code>capacidade_excedida</code> , <code>aguardando_parametro</code> , <code>estimativa_indisponivel</code> , <code>continuo_acima_maxima</code> , <code>continuo_abaixo_minima</code> , <code>lote_sem_piquete</code> , <code>sem_dia_de_manejo</code> , <code>passando_do_ponto</code>	Tipo do aviso
data	date	"2026-10-01"	Dia a que se refere
texto	str	texto	Frase pronta para o produtor
confianca / motivo_confianca	Confianca / str	texto	Como na movimentação
piquete_id / lote_id	UUID None	texto ou null	A quem se refere

PedidoValidacao: pedido de medir o capim com régua

Campo	Tipo	Em JSON	Significado
piquete_id / piquete_nome	UUID / str	texto	Piquete a medir
motivo	str	texto	Frase pronta: por que o sistema está pedindo

ResumoPiquete: a fotografia de cada piquete (usada no mapa)

Campo	Tipo	Em JSON	Significado
piquete_id / nome	UUID / str	texto	Piquete
situacao	SituacaoPiquete	"ocupado" "descansando"	Tem gado ou está descansando
lote_atual_nome	str None	texto ou null	Lote que está nele
altura_hoje_cm	float None	número ou null	Altura estimada hoje; null = sem estimativa
altura_entrada_alvo_cm / altura_saida_alvo_cm	float None	número ou null	Alvos (nulos no contínuo ou sem parâmetro)
confianca / motivo_confianca	Confianca / str	texto	Confiança da estimativa do piquete
faltantes	tuple[str, ...]	lista de texto	O que falta para estimar ou planejar; vazio = tudo certo

2. Stack e onde fica o seu código

Camada	Tecnologia
Backend	Python 3.12, FastAPI, psycopg 3 (SQL puro, sem ORM), Pydantic
Banco e login	Supabase (PostgreSQL + PostGIS + Auth)
Frontend	React + TypeScript + Vite, react-router, @supabase/supabase-js, CSS puro com variáveis
Hospedagem	Render (API) · Vercel (site) · GitHub Actions (rotina de hora em hora)

Arquivos que são seus (só você edita):

- Backend: `src/seugado/api/rotas_fazenda.py`, `rotas_lotes.py`, `rotas_plano.py` (já existem, vazios); `src/seugado/cadastro/fazenda.py`, `cadastro/clientes.py`, `cadastro/lotes.py`; `src/seugado/jobs/ciclo.py`.
- Frontend: `frontend/src/paginas/Login.tsx`, `Clientes.tsx`, `Configuracoes.tsx` (a página “Fazenda”), `Lotes.tsx`, `Plano.tsx`; `frontend/src/estilo/*`; componentes seus em `frontend/src/componentes/ui/`.
- Infra: `.github/workflows/rotina.yml`, `render.yaml`.
- Testes: `tests/cadastro/test_fazenda.py`, `test_clientes.py`, `test_lotes.py`, `tests/jobs/*`.

Você usa, mas não edita: `api/deps.py` (Conexao), `api/auth.py` (Usuario, FazendaAutorizada), `persistencia/*`, `contratos.py`, `planner/estado.py`, `frontend/src/lib/*` (`api()`, `supabase`, `tipos`,

`useFazenda()`), `frontend/src/App.tsx` e `frontend/src/componentes/Layout.tsx` (as rotas e o seletor de fazenda já existem). **Não adicione dependências.**

O Ezequiel também faz frontend (a página Mapa). Vocês dois usam o mesmo `tema.css`: mande a ele o link do Figma e as cores logo cedo.

3. Preparar o ambiente

Guia completo de instalação e comandos (Windows e Mac/Linux, token para o /docs, testes, Git e problemas comuns): `docs/equipe/COMO-RODAR.md`. Não rode migrações nem nada no Supabase: o banco já está pronto e é compartilhado.

Importante: o Python não lê o `.env` sozinho. Todo comando `uv run local` leva `--env-file .env` (como nos exemplos abaixo); sem isso a API responde erro 500 e os testes de banco são pulados.

1. Instale Git, **Node 20+** e **uv** (`docs.astral.sh/uv`). Clone e crie a branch `feat/leandro-admin-rotina`.
2. Raiz: copie `.env.example` para `.env` e preencha com o que o Kauê mandar em privado: `DATABASE_URL`, `SUPABASE_URL`, `SUPABASE_ANON_KEY` e as duas do Earth Engine. **Nunca commite o .env.**
3. `frontend/.env` a partir do `.env.example`: `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_API_URL=http://localhost:8000`.
4. Backend: `uv sync --group dev` e `uv run --env-file .env uvicorn seugado.api.main:app --app-dir src --reload` (docs em `http://localhost:8000/docs`).
5. Frontend: `cd frontend && npm install && npm run dev` (`http://localhost:5173`).
6. Peça ao Kauê a sua conta de admin (e-mail e senha).

Antes do PR: `uv run ruff check .`, `uv run --env-file .env pytest` e `npm run build` sem erros.

4. A regra de gravação do projeto

- **Cliente e fazenda** são cadastro de configuração: INSERT/UPDATE direto nas tabelas `cliente` e `fazenda` (ADR-022).
- **Lote** (e as movimentações que você registra) é história: vira **evento**.

```
from datetime import UTC, datetime
from seugado.core.models import OrigemEvento, TipoEvento
from seugado.persistencia.eventos import registrar_evento
from seugado.persistencia.projecao_db import reconstruir_projecao

agora = datetime.now(UTC)
registrar_evento(conn, fazenda_id, TipoEvento.LOTE_CRIADO, OrigemEvento.PRODUTOR,
                 ocorrido_em=agora, payload={...}, ator=str(usuario.id))
registrar_evento(conn, fazenda_id, TipoEvento.MANEJO_CONFIRMADO, OrigemEvento.PRODUTOR,
```

```

        ocorrido_em=agora, payload={...}, ator=str(usuario.id))
reconstruir_projecao(conn, fazenda_id)  # atualiza estado_lote / estado_piquete
conn.commit()                          # a ROTA faz o commit, uma vez, no fim

```

Importante: `ocorrido_em` é **sempre o momento atual** (`datetime.now(UTC)`). A data real do fato (“o lote está lá desde dia 20”) vai **dentro do payload** (`data_execucao`). A reconstrução processa os eventos pela ordem de `ocorrido_em`; se você datar a movimentação no passado, ela é processada antes de o lote existir e dá erro.

A conexão (`Conexao`) **não faz commit sozinha**. Para ler como dicionário, use `conn.cursor(row_factory=dict_row)` (from `psycpg.rows` import `dict_row`).

Tabelas que você usa:

Tabela	Uso
cliente	id, nome, telefone, observacoes, criado_em
fazenda	id, cliente_id, nome, timezone, funcionarios_disponiveis, animais_por_funcionario_dia, dias_preferenciais_manejo (smallint[]), envio_plano_dia, envio_plano_hora, ultimo_envio_semanal, ultima_rotina_diaria, telegram_chat_id, codigo_vinculo_telegram, ativo
estado_lote (derivada, só leitura)	lote_id, nome, indissoluvél, composicao (jsonb), piquete_atual_id, desde, peso_vivo_total_kg
estado_piquete (derivada, só leitura)	piquete_id, nome, ativo, lote_atual_id
plano	escrito pelo João; você lê por <code>carregar_plano_atual</code>

Payloads dos eventos de lote (todos os campos obrigatórios):

```

# lote_criado
{"entidade_id": "<uuid do lote>", "nome": "Recria", "indissoluvél": False,
 "composicao": [{"categoria": "novilho", "n_animais": 150, "peso_medio_kg": 337.5,
                  "origem_peso": "ua_tabela"}]} # categoria:
bezerro|bezerra|novilho|novilha|vaca|boi|touro
# lote_alterado: os mesmos campos + "ativo": True
# lote_dissolvido
{"entidade_id": "<uuid do lote>"}
# manejo_confirmado (onde o lote está / correção de posição)
{"entidade_id": "<uuid novo>", "lote_id": "<uuid do lote>",
 "piquete_destino_id": "<uuid do piquete>", "data_execucao": "2026-09-20"}

```

5. Contratos da sua parte

5.1 Rotas

Todas exigem login (`Usuario`). As que têm `{fazenda_id}` usam `FazendaAutorizada` (404 se a fazenda não existe).

Método e caminho	Corpo	Resposta
GET /me	—	{ "usuario_id", "email" }
GET /clientes	—	Cliente[] (por nome)
POST /clientes	ClienteIn	201 + Cliente
PUT /clientes/{cliente_id}	ClienteIn	Cliente
GET /fazendas	—	Fazenda[] (por nome do cliente, depois nome da fazenda; só ativas)
POST /fazendas	FazendaIn	201 + Fazenda
GET /fazendas/{fazenda_id}	—	Fazenda
PUT /fazendas/{fazenda_id}	FazendaIn	Fazenda
GET /fazendas/{fazenda_id}/teleg ram	—	{ "vinculado": bool, "link": "https://t.me/<bot>?start=<codigo>" }
GET /fazendas/{fazenda_id}/lotes	—	Lote[] (por nome)
POST /fazendas/{fazenda_id}/lotes	LoteIn	201 + Lote
PUT /fazendas/{fazenda_id}/lotes /{lote_id}	LoteIn	Lote
DELETE /fazendas/{fazenda_id}/lotes /{lote_id}	—	204
GET /fazendas/{fazenda_id}/plano /atual	—	PlanoManejo (vigente) ou 404
POST /fazendas/{fazenda_id}/ciclo	{ "ingerir_satelite": true, "enviar": true }	PlanoManejo

```
// ClienteIn / Cliente (Cliente = ClienteIn + "id")
{ "nome": "João da Silva", "telefone": "+55 82 99999-0000", "observacoes": null }

// FazendaIn / Fazenda (Fazenda = FazendaIn + "id" + "cliente_nome")
{ "cliente_id": "...", "nome": "Fazenda Boa Vista", "timezone": "America/Fortaleza",
  "funcionarios_disponiveis": 2, "animais_por_funcionario_dia": 150,
  "dias_preferenciais_manejo": [0, 3], // segunda e quinta
  "envio_plano_dia": 6, "envio_plano_hora": 18 } // domingo às 18h

// LoteIn
{ "nome": "Recria", "indissoluvel": false,
  "composicao": [ { "categoria": "novilho", "n_animais": 150, "peso_medio_kg": null } ],
  "piquete_atual_id": "...", // obrigatório
  "desde": "2026-09-25" } // opcional; padrão: hoje
```

Os tipos TypeScript (Cliente, Fazenda, Me, Lote, ComposicaoItem, PlanoManejo...) **já existem** em frontend/src/lib/tipos.ts. O PlanoManejo tem exemplo completo em tests/fixtures/plano_exemplo.json: use-o para montar a página do plano antes de a integração estar pronta.

5.2 Funções que a sua rotina chama (de outras pessoas)

```
from seugado.sensing.ingestao import ingerir_leituras           # Kauê
from seugado.persistencia.projecao_db import reconstruir_projecao # Kauê
from seugado.planner.carga import montar_estado_projetado      # Kauê
from seugado.planner.otimizador import gerar_plano             # João
from seugado.planner.comparacao import comparar_planos         # João
from seugado.persistencia.planos import (salvar_plano, carregar_plano_atual,
                                         ids_respondidos)       # João
from seugado.delivery.envio import enviar_plano, avisar_plano_candidato # Leo
from seugado.delivery.confirmacao import lembrar_pendentes     # Leo

NOVAS
    ingerir_leituras(conn, fazenda_id, data_inicio, data_fim) -> int      # quantas leituras

    reconstruir_projecao(conn, fazenda_id) -> EstadoFazenda
    montar_estado_projetado(conn, fazenda_id, data_base: date) -> EstadoProjetado
    gerar_plano(estado, fazenda: Fazenda, agora: datetime, plano_id: UUID) -> PlanoManejo
    comparar_planos(anterior, novo, respondidas: frozenset[UUID], hoje: date) ->
tuple[DiferencaLote, ...]
    salvar_plano(conn, plano, status: str = "vigente") -> None           # "vigente" |
    "candidato"
    carregar_plano_atual(conn, fazenda_id) -> PlanoManejo | None
    ids_respondidos(conn, fazenda_id) -> frozenset[UUID]
    enviar_plano(conn, plano, atualizado: bool = False) -> bool
    avisar_plano_candidato(conn, plano, diferencas: tuple[DiferencaLote, ...]) -> bool
    lembrar_pendentes(conn, fazenda_id, hoje: date) -> int
```

Nenhuma delas faz commit: **a rotina é que faz.**

Módulos que ainda não estão na sua branch. As funções do João e do Leo só chegam na `main` na integração de segunda, e as do Kauê (`ingestao`, `carga`) ao longo de domingo. Por isso, **importe-as dentro das funções** de `jobs/ciclo.py` e `rotas_plano.py` (não no topo do arquivo), e nos testes injete módulos falsos com `monkeypatch.setitem(sys.modules, "seugado.planner.otimizador", modulo_falso)`.

6. Tarefas (na ordem)

L0: Backend base: `/me`, clientes e fazendas (PR pequeno, primeiro)

Por quê: o site inteiro depende disso. Sem `GET /me` e `GET /fazendas`, o seletor de fazenda fica vazio e ninguém (inclusive o Ezequiel) consegue usar as páginas. É a única tarefa da equipe que bloqueia outra pessoa.

Como:

- `cadastro/clientes.py`: `listar_clientes`, `criar_cliente`, `atualizar_cliente` (SQL direto na tabela `cliente`; nome não vazio).
- `cadastro/fazenda.py`:
 - `listar_fazendas(conn)`: `fazenda + cliente.nome AS cliente_nome` (LEFT JOIN), só ativo;
 - `criar_fazenda(conn, dados)` -> UUID (INSERT ... RETURNING id);
 - `carregar_fazenda(conn, fazenda_id)` -> Fazenda (a dataclass de `seugado.core.models`, com `dias_preferenciais_manejo` como `tuple[int, ...]`). A rotina usa esta função;
 - `atualizar_fazenda(conn, fazenda_id, dados)`.

3. Validação (Pydantic):

- `nome` não vazio; `cliente_id` existente;
- `funcionarios_disponiveis` ≥ 1 e `animais_por_funcionario_dia` ≥ 1 ;
- `dias_preferenciais_manejo` com pelo menos 1 valor, todos entre 0 e 6, sem repetição (ordene);
- `envio_plano_dia` entre 0 e 6; `envio_plano_hora` entre 0 e 23;
- `timezone` padrão `America/Fortaleza`.

4. Rotas GET `/me` (`{"usuario_id", "email"}` de `Usuario`), `/clientes` e `/fazendas` (tabela 5.1). Cada rota que escreve termina com `conn.commit()`.

5. Abra o PR `feat/leandro-base` e **avise o Kauê para fazer o merge na hora**. Depois avise o Ezequiel para dar `git pull`.

Critérios de aceite:

- [] Com o token de uma conta da equipe, GET `/me` e GET `/fazendas` respondem 200.
- [] Criar cliente e fazenda pelo `/docs` faz a fazenda aparecer no seletor do site.
- [] Merge na `main` até as 10h.

L1: Identidade visual (Figma) + `tema.css` (timebox: 1 hora)

Por quê: na terça, a banca vai julgar também o produto. Paleta e tipografia consistentes fazem o painel parecer um produto só. Lembre que quem usa o painel é a equipe; o produtor vê só o Telegram.

Como:

1. No Figma, wireframes simples das telas: Login, Clientes e fazendas, Fazenda, Mapa, Lotes, Plano da semana.
2. Paleta (verde de pasto como primária; cores de estado do mapa: verde, amarelo, azul, vermelho, cinza), tipografia (fonte do sistema) e raio de borda.
3. Passe para `frontend/src/estilo/tema.css`: troque os valores das variáveis que já existem e acrescente `--cor-pronto`, `--cor-crescendo`, `--cor-ocupado`, `--cor-sair` e `--cor-sem-estimativa`.
4. Mande o link e as variáveis ao Ezequiel.

Critérios de aceite:

- [] Telas no Figma e link compartilhado.
- [] Nenhuma cor escrita direto nas páginas.

Fora do escopo: logo definitivo; biblioteca de componentes; modo escuro.

L2: Login da equipe (`Login.tsx`)

Por quê: o painel mostra dados de todos os clientes; só a equipe entra.

Como:

1. Formulário de e-mail e senha com **um único botão, Entrar** (`supabase.auth.signInWithPassword`). **Não há “Criar conta”**.

2. Sucesso → `navigate("/mapa")`. Erros em português (“E-mail ou senha incorretos”).
3. No Supabase (o Kauê é o dono do projeto, combine com ele):
 - *Authentication* → *Sign In / Providers*: **desligue “Allow new users to sign up”**;
 - crie as contas da equipe em *Authentication* → *Users* → *Add user*;
 - *URL Configuration*: Site URL = endereço do Vercel; Redirect URLs com `http://localhost:5173`.

Critérios de aceite:

- [] Conta da equipe entra; e-mail desconhecido não entra.
- [] Sair e entrar de novo funciona.

Fora do escopo: login social; recuperação de senha; papéis diferentes entre admins.

L3: Páginas “Clientes e fazendas” (`Clientes.tsx`) e “Fazenda” (`Configuracoes.tsx`)

Por quê: é por aqui que a equipe cadastra o cliente e a **rotina** da fazenda que o otimizador respeita (R7 e R8 do motor, o diferencial do SeuGado), e que o produtor recebe o link do bot.

Como (Clientes e fazendas):

- Lista de clientes, cada um com as suas fazendas embaixo.
- “Novo cliente” (nome, telefone, observações) e “Nova fazenda” dentro de um cliente.
- Clicar numa fazenda → `selecionar(id)` (de `useFazenda()`) → vai para `/mapa`.
- Formulário da fazenda:
 - **Nome da fazenda;**
 - **“Quantas pessoas fazem o manejo do gado?”** (`funcionarios_disponiveis`);
 - **“Quantos animais cada pessoa consegue mover sozinha num dia?”** (`animais_por_funcionario_dia`);
 - **“Em quais dias da semana se pode mexer no gado?”**: 7 caixas seg...dom, com segunda e quinta marcadas por padrão;
 - **“Quando o produtor quer receber o plano?”**: dia da semana (select) + hora (0–23).

Como (Fazenda, a página da fazenda selecionada):

- O mesmo formulário (`PUT /fazendas/{id}`) + nome do cliente.
- Cartão **Telegram**:
 - não vinculado: “Mande este link para o produtor abrir no celular e tocar em **Começar**”, com o link e os botões “Copiar” e “Atualizar status”;
 - vinculado: “Telegram conectado ✓”.
- Depois de salvar, chame `recarregar()` para o seletor refletir o nome novo.

Critérios de aceite:

- [] Cadastrar cliente + fazenda + rotina e ver a fazenda no seletor.
- [] Mudar dias de manejo e horário de envio persiste.
- [] O link do Telegram aparece e o status muda para vinculado depois do `/start` no bot.

Fora do escopo: apagar cliente ou fazenda; importar planilha; mapa aqui (é do Ezequiel).

L4: Lotes: backend e página

Por quê: o consumo do lote é metade da conta do otimizador: quanto o lote come define em quantos dias ele rapa o piquete. E o otimizador precisa saber **onde cada lote está agora e quantos animais** tem (a mão de obra é contada em animais).

Como (backend, cadastro/lotes.py + rotas_lotes.py):

1. `criar_lote(conn, fazenda_id, ator, nome, indissoluvél, composicao, piquete_atual_id, desde) -> UUID:`
 - valide:
 - nome não repetido entre os lotes da fazenda;
 - composição com pelo menos 1 item, categorias sem repetição, `n_animais >= 1`;
 - peso_medio_kg, se informado, entre 20 e 1.500;
 - o piquete existe, está ativo e **está vazio** (`estado_piquete.lote_atual_id IS NULL`);
 - `desde <= hoje`;
 - peso ausente → `peso_por_ua_kg(CategoriaAnimal(categoria))` com `origem_peso: "ua_tabela"`; informado → `"produtor"`;
 - registre `LOTE_CRIADO` e em seguida `MANEJO_CONFIRMADO` (`novo entidade_id, lote_id, piquete_destino_id = piquete_atual_id, data_execucao = desde`), ambos com `ocorrido_em = agora`;
 - `reconstruir_projecao`.
2. `editar_lote(...): LOTE_ALTERADO` (todos os campos + `ativo: True`). Se o `piquete_atual_id` mudou (correção feita pela equipe), registre também `MANEJO_CONFIRMADO` para o novo piquete (vazio) com `data_execucao = hoje`.
3. `dissolver_lote(...): LOTE DISSOLVIDO`.
4. `listar_lotes(conn, fazenda_id): estado_lote + nome do piquete; n_animais_total = soma`.
5. Rotas: `ValueError/ValidationError` → 400, lote inexistente → 404, piquete ocupado → 409. `conn.commit()` no fim.

Como (frontend, Lotes.tsx):

- Lista em cartões:

Recria · 150 animais · Piquete 2 desde 25/09 · 50,6 t de peso vivo

- Formulário:
 - **Nome**;
 - **Composição**: linhas com categoria (bezerro, bezerra, novilho, novilha, vaca, boi, touro), quantidade e peso médio (kg), com a dica *“Sem peso? Deixe em branco: estimamos pela tabela de Unidade Animal”*, e o botão *“+ categoria”*;
 - **Onde o lote está agora**: piquetes livres de `GET /fazendas/{id}/piquetes`, mais o atual do lote na edição;
 - **Desde** (data);
 - **Lote indissolúvel**, com a explicação.

- Sem piquete cadastrado → “Cadastre os piquetes no Mapa antes dos lotes”.

Critérios de aceite:

- [] “Recria: 150 novilhos, sem peso, no Piquete 2” grava 337,5 kg com `origem_peso: "ua_tabela"`, e o Piquete 2 aparece ocupado.
- [] 30 touros sem peso → 562,5 kg.
- [] Não deixa dois lotes no mesmo piquete.

Fora do escopo: raça, brinco, pesagem individual; fusão de lotes; histórico na tela.

L5: Plano da semana (`rotas_plano.py` + `Plano.tsx`)

Por quê: é onde a equipe confere o que o produtor recebeu e dispara um plano na hora. Na terça, esta página é a demo.

Como (backend):

1. GET `/fazendas/{id}/plano/atual`: `carregar_plano_atual` (plano **vigente**) → `plano_para_dict`, ou 404 “Ainda não há plano”.
2. POST `/fazendas/{id}/ciclo`: `executar_ciclo(conn, fazenda_id, ingerir_satelite=corpo.ingerir_satelite, enviar=corpo.enviar)` → plano. Pode levar 1-2 min. Erros de serviço externo → 502 com a mensagem.

Como (frontend, `Plano.tsx`):

- Cabeçalho:
 - “Plano de seg 28/09 a dom 04/10 · gerado em 28/09 às 05:00”;
 - botão **Gerar e enviar plano agora**, com spinner e “Buscando satélite e clima...”;
 - uma caixa “enviar ao produtor”, marcada por padrão.
- Movimentações agrupadas por dia (só dias com movimento). Em cada cartão:
 - título **Mover Recria: Piquete 2 → Piquete 6**;
 - `motivo`;
 - “Previsão: N dias no piquete”;
 - selo de confiança e `motivo_confianca`.
- Alertas (ícone por `tipo` + `texto`) e medições pedidas (`pedidos_validacao`), com campo “cm” que chama `POST /fazendas/{id}/piquetes/{piquete_id}/alturas` (rota do Ezequiel).
- Sem plano (404): estado vazio com “Gerar o primeiro plano”.
- Não recalcula nada no frontend.

Critérios de aceite:

- [] Com `plano_exemplo.json`: segunda com 2 movimentos, quinta com 1, 2 alertas e 1 pedido de medição.
- [] O botão gera o plano novo e a página atualiza.

Fora do escopo: editar o plano; responder por ele (isso é pelo Telegram).

L6: A rotina (jobs/ciclo.py)

Por quê: é o que faz o SeuGado rodar **sozinho**. O cálculo roda **todo dia** (o satélite passa a cada ~4-5 dias, sem horário fixo), o plano semanal chega **no dia e hora que o produtor escolheu**, e o produtor só é incomodado no meio da semana se algo que ele ainda não fez mudou.

Como:

```
HORA_ROTINA_DIARIA = 6  # hora local; escolha nossa

def calcular_plano(conn, fazenda: Fazenda, hoje: date, ingerir_satelite: bool) ->
tuple[PlanoManejo, int]:
    novas = ingerir_leituras(conn, fazenda.id, hoje - timedelta(days=30), hoje) if
ingerir_satelite else 0
    reconstruir_projecao(conn, fazenda.id)
    conn.commit()  # leituras salvas mesmo se algo falhar
    depois
    estado = montar_estado_projetado(conn, fazenda.id, hoje)
    return gerar_plano(estado, fazenda, datetime.now(UTC), uuid4()), novas

def executar_ciclo(conn, fazenda_id, hoje=None, ingerir_satelite=True, enviar=True,
atualizado=False) -> PlanoManejo:
    """Calcula e JÁ coloca em vigor (botão da web e recálculo pedido pelo produtor no
bot)."""
    fazenda = carregar_fazenda(conn, fazenda_id)
    hoje = hoje or datetime.now(ZoneInfo(fazenda.timezone)).date()
    plano, _ = calcular_plano(conn, fazenda, hoje, ingerir_satelite)
    salvar_plano(conn, plano, "vigente")
    conn.commit()
    if enviar:
        enviar_plano(conn, plano, atualizado=atualizado)
    return plano

def entregar_plano_semanal(conn, fazenda_id, hoje) -> PlanoManejo:
    plano = executar_ciclo(conn, fazenda_id, hoje, ingerir_satelite=True, enviar=True)
    # UPDATE fazenda SET ultimo_envio_semanal = hoje, ultima_rotina_diaria = hoje; commit
    return plano

def rotina_diaria(conn, fazenda_id, hoje) -> None:
    lembrar_pendentes(conn, fazenda_id, hoje)  # bot pergunta o que ficou sem
    resposta
    fazenda = carregar_fazenda(conn, fazenda_id)
    vigente = carregar_plano_atual(conn, fazenda_id)
    novo, novas = calcular_plano(conn, fazenda, hoje, ingerir_satelite=True)
    # UPDATE fazenda SET ultima_rotina_diaria = hoje; commit
    if vigente is None or novas == 0:
        return  # sem plano em vigor ou sem imagem
    nova
    difs = comparar_planos(vigente, novo, ids_respondidos(conn, fazenda_id), hoje)
    if difs:
        salvar_plano(conn, novo, "candidato")
        conn.commit()
        avisar_plano_candidato(conn, novo, difs)  # "quer ver as mudanças?"

def executar_agenda(agora_utc: datetime | None = None) -> int:
    """Roda de hora em hora. Devolve 0 se tudo certo, 1 se alguma fazenda falhou."""
```

executar_agenda:

1. Abre a própria conexão (psycopg.connect(os.environ["DATABASE_URL"])).

2. Para cada fazenda `ativo` com pelo menos 1 piquete ativo:

- `local = agora.astimezone(ZoneInfo(timezone))`, `hoje = local.date()`;
- **entrega semanal** se `local.weekday() == envio_plano_dia`, `local.hour >= envio_plano_hora` e `ultimo_envio_semanal != hoje`;
- senão, **rotina diária** se `local.hour >= HORA_ROTINA_DIARIA` e `ultima_rotina_diaria != hoje`.

(O `>=` com a data de controle garante que um atraso do GitHub não faz perder o horário, e que nada roda duas vezes no mesmo dia.)

3. Cada fazenda roda num `try`: erro → `rollback()`, imprime com o id da fazenda e **continua**.

Linha de comando (`python -m seugado.jobs.ciclo`, com `argparse`):

- `--agenda`;
- `--fazenda <uuid>` COM `--entregar`, `--diaria` OU `--recalcular` (este último = `executar_ciclo`);
- `--sem-satelite` e `--sem-envio`.

Critérios de aceite:

- [] Testes com funções falsas conferem:
 - a ordem das chamadas em cada rotina;
 - que o candidato só é salvo e avisado quando há leitura nova e diferença;
 - que a entrega semanal não se repete no mesmo dia;
 - que uma fazenda com erro não derruba as outras.
- [] `--fazenda <id> --recalcular --sem-satelite` não chama o Earth Engine.

Fora do escopo: fila, paralelismo, agendamento dentro da API, lembrete em outro horário.

L7: Deploy

Por quê: “funcionando de verdade” na terça quer dizer acessível pela internet e rodando sem o computador de ninguém ligado.

Como:

1. **API no Render** (grátis), *New* → *Web Service* a partir do repositório:

- `PYTHON_VERSION=3.12`;
- Build: `pip install uv && uv sync --frozen --no-dev`;
- Start: `uv run uvicorn seugado.api.main:app --app-dir src --host 0.0.0.0 --port $PORT`;
- Health check: `/saude`;
- Variáveis: `DATABASE_URL`, `SUPABASE_URL`, `SUPABASE_ANON_KEY`,
`SEUGADO_CORS_ORIGINS=https://<app>.vercel.app,http://localhost:5173`, `SEUGADO_GEE_PROJECT`,
`SEUGADO_GEE_SERVICE_ACCOUNT_JSON`, `TELEGRAM_BOT_TOKEN`, `TELEGRAM_BOT_USERNAME`,
`TELEGRAM_WEBHOOK_SECRET` (as três últimas do Leo).
- Guarde em `render.yaml`.
- O plano grátis dorme após 15 min sem uso. **Antes da apresentação, abra** `/saude`.

2. **Site no Vercel:** root `frontend`, framework Vite, variáveis `VITE_SUPABASE_URL`,
`VITE_SUPABASE_ANON_KEY`, `VITE_API_URL=https://<api>.onrender.com`.

3. Rotina no GitHub Actions, `.github/workflows/rotina.yml`:

- `on.schedule.cron: "7 * * * *"` (de hora em hora, no minuto 7) + `workflow_dispatch` com os inputs `fazenda_id` (opcional) e `acao` (`agenda` | `entregar` | `diaria` | `recalcular`);
- `passos`: `actions/checkout`, `astral-sh/setup-uv` (versão estável atual), `uv sync --frozen --no-dev`, `uv run python -m seugado.jobs.ciclo --agenda` (ou a ação escolhida), com `PYTHONPATH: src`;
- `secrets`: `DATABASE_URL`, `SEUGADO_GEE_PROJECT`, `SEUGADO_GEE_SERVICE_ACCOUNT_JSON`, `TELEGRAM_BOT_TOKEN`.

4. Mande à equipe as URLs do site e da API (o Leo precisa da URL da API para o webhook).

Critérios de aceite:

- [] O site no Vercel faz login e fala com a API no Render.
- [] “Run workflow” com `entregar` gera o plano, que aparece no site e no Telegram.
- [] Nenhuma chave no repositório.

Fora do escopo: domínio próprio; homologação separada; monitoramento.

7. Integração na segunda (a sua parte é a cola)

1. Merge na ordem: Ezequiel → você (base, clientes, fazendas e lotes) → João → Leo → você (rotina e deploy).
2. Na web: login → cliente → fazenda → piquetes (Ezequiel) → lotes → “Gerar e enviar plano agora” → o plano aparece aqui, no mapa e no Telegram.

8. Usando IA no seu fluxo

Dê à IA este documento e o arquivo que ela vai editar, **uma tarefa por vez**. Exemplo:

“Leia docs/equipe/LEANDRO.md, seções 4, 5 e a tarefa L6. Implemente `src/seugado/jobs/ciclo.py` exatamente como descrito, importando as funções de outras pessoas dentro das funções. Não altere outros arquivos nem adicione dependências.”

Se a IA quiser mudar arquivo de outra pessoa, instalar pacote ou alterar contrato, a resposta é não: fale com o Kauê.